

# Supplemental Material: Providing Rapid Design Feedback for 3D Obstacle Course Games Using Constrained Solvability Queries

ANONYMOUS AUTHOR(S)

SUBMISSION ID: 2451

## ACM Reference Format:

Anonymous Author(s). 2026. Supplemental Material: Providing Rapid Design Feedback for 3D Obstacle Course Games Using Constrained Solvability Queries. 1, 1 (May 2026), 11 pages. <https://doi.org/10.1145/nnnnnnnn>. nnnnnnnn

## 1 Solve Rate comparisons

Fig. 1 compares solve rate for each of the methods compared with average solution time in Fig. 3 of the main paper. To compute the solve rate, we run each of our 56 queries over 5 seeds and consider fraction of seeds solved under 90s as per-query solve rate. For each bucket, we then find the mean and inter-quartile range (top/bottom 25%-ile) over all the per-query solve rates and report in Fig. 1. Both GoExplore (GE) and GoExplore-PolicySelection (GE-PS) have a high solve rate ( $> 90\%$ ) over both short and medium queries, with GoExplore solving all of them. Cell-Free Latent GoExplore (CFLGE) has a much lower solve rate of 58% and 5% on short and medium buckets respectively. We observe the main advantages of the policy selection in the long regime where GoExplore could solve only 31% of the queries whereas Policy-Selection solves 20% more queries. Policy-Based GoExplore (PBGE) consistently could not solve any query. We suspect this is because the time it takes to train a policy to return to previous bins (and simulate the returning actions) is too costly for a 90s time budget. Pretrained RL policy has 50% solve rate on short and medium queries and dips to 21% on long queries.

## 2 Policy-Selection GoExplore

### 2.1 Background: The GoExplore Algorithm (+ our modifications)

GoExplore [Ecoffet et al. 2021] is a search-based framework designed for hard-exploration problems. We refer the reader to Alg. 1 for full implementation of GoExplore, along with our proposed modifications using the pretrained policy highlighted in red. GoExplore addresses the limitations of reinforcement learning algorithms in exploration of the space by separating the search for new states from the act of returning to them, thereby avoiding detachment (losing track of exploration frontiers) and derailment (stochastic noise preventing returns to distant states). When enabled with parallel simulations, GoExplore can return a solution trajectory within a few seconds, which makes it suitable for our use case.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM XXXX-XXXX/2026/5-ART

<https://doi.org/10.1145/nnnnnnnn>. nnnnnnnn

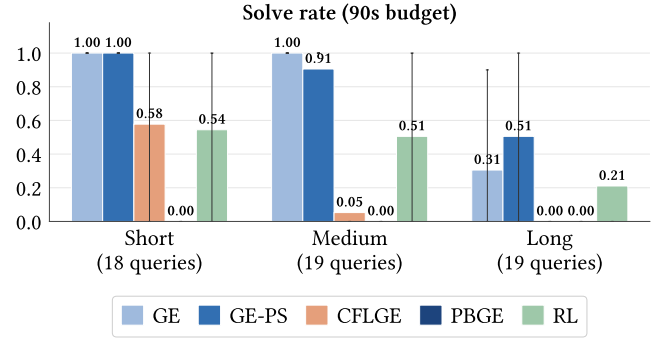


Fig. 1. Solve rate by Time to First Solution across the five methods: GoExplore [Ecoffet et al. 2021] (GE), GoExplore-PolicySelection (GE-PS), Cell-Free Latent GoExplore [Gallouédec and Dellandréa 2023] (CFLGE), Policy-Based GoExplore [Ecoffet et al. 2021] (PBGE), and Reinforcement Learning [Schulman et al. 2017] (RL) based with queries grouped into Short / Medium / Long buckets based on GoExplore's mean solve time.

**Core Exploration Loop** The algorithm maintains an archive of “bins” representing different states. The exploration proceeds through a continuous loop:

- **Selection:** A bin is chosen from the archive according to a sampling heuristic.
- **Return:** The agent returns to the exact state of that bin without added exploration by resetting a deterministic simulator.
- **Explore:** The agent explores from that state for a fixed duration using random or intelligent actions.
- **Update:** New states are added to the archive, existing bins are updated if a better trajectory (higher score or shorter length) is discovered, and statistics related to the traversed bins are updated.

### 2.2 Binning and Bin Representation

To handle high-dimensional state spaces like pixel space observations, GoExplore uses a binning function to map states into discrete bins. Given a level, we construct a 6-dimensional binning function based on: the agent's XYZ position, the current timestep, the number of jumps the agent has remaining, and the position of a single interactable object (i.e. a block the agent can push). We allocate 32 bins each to the horizontal dimensions, and 10 to the vertical dimension. Allocation of bins to time, jump remaining, and interactable object position is dependent on whether the relevant constraints are present in the level. See Alg. 1 below.

### 2.3 Scoring Function and Bin Sampling Criteria

We use the same scoring function as the original GoExplore algorithm, which is the number of the steps in a trajectory (lower is better). This scoring function prioritizes finding faster paths to the goal. Bins are selected from the archive using a weight inversely proportional to the number of times that bin has been seen during exploration (Lines 31–34 in Alg. 1).

### 2.4 Policy Selection

GoExplore [Ecoffet et al. 2021] chooses actions at random with a 95% probability of repeating the previous action. We call this the default random exploration. We modify GoExplore’s explore phase to explore using either our pretrained policy or the default random exploration. Specifically, when a bin is selected to explore from for the first time after being discovered or updated, we use our pretrained policy for the duration of the episode (up to the length of the exploration step). For the subsequent selections of this bin, we use the default random exploration. This heuristic of using the pretrained policy only once for a new bin is based on two observations: a) our environments are deterministic up to collisions resolution in the rigid body simulator, and b) the highest performing policy (at the end of pretraining) outputs nearly deterministic probability distributions over actions even in novel environments. These two observations together suggest the benefit from a pretrained policy can be had with a single rollout from each new state (which in practice might be multiple rollouts if many worlds sample this bin simultaneously). We give a detailed pseudocode of our Policy-Selection GoExplore algorithm in Alg. 1.

### 2.5 Sampling for diverse solutions

When we find a trajectory, we reset the checkpoint of any bin that is storing a trajectory starting from the found trajectory’s starting point. This ensures both a) we never return two trajectories with exact sectional overlap, and b) we target resetting the search in areas effected by the found solution trajectory. This way other areas, dealing with potentially more complex trajectories, are less likely to be effected.

## 3 RL Pre-Training Details

This section described our procedure for training the agent used in our implementation of policy-selection GoExplore.

### 3.1 Overview

We model the task of navigating the obstacle courses as a stochastic Markov decision process. Formally,  $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$ , where  $\mathcal{S}$  is the state space of the game,  $\mathcal{A}$  is the agent’s action space,  $\mathcal{P}$  is the transition probability function between game states given the agent’s action, and  $\mathcal{R}$  is the reward function that provides a real-valued score for each action chosen by the agent. We train our agents using reinforcement learning, which aims to produce a *policy*  $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$  that maximizes the expected cumulative reward  $J(\pi) = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t \mathcal{R}(s_t, a_t) \mid s_0, \pi \right]$ , where  $s_t, a_t$  are the state and action at time step  $t$  and  $\gamma$  is a discount factor on future rewards.

### 3.2 Implementation

Our training process timesteps 8192 worlds in parallel. At the start of training, we first sample a set of 1024 obstacles uniformly at random from our obstacle DSL and upload them to GPU memory. The obstacle type (e.g. Lava Ring Drop, Lava Stairs) and parameter values (e.g. platform length, lava width) are chosen uniformly at random (see Sec. 5). All 8192 worlds in parallel then sample from this set to begin a new episode. After 100 policy updates (where each update occurs after 40 batch steps), we sample a new set of 1024 obstacles. We found that updating the 1024 obstacle set every 100 policy updates struck a good balance between ensuring a wide range of obstacles for training and avoiding the GPU-CPU sync incurred by updating the GPU memory buffer. We do not upload more than 1024 obstacles at a time to GPU memory because we wish to maximize the amount of GPU memory available for agent training.

The agent policies are based on the actor-critic model, consisting of a fully connected Multi Layer Perceptron (MLP) with three hidden layers of width 256 and separate heads for the actor and critic.

### 3.3 Agent Observations and Reward Function

The full agent observation space is as follows:

- Raytraced Observations: 128 rays traced in a spherical fibonacci distribution centered on the agent. These rays are traced to a maximum distance of 200m. Hits return distance to object and a one-hot encoding indicating whether the object is harmful. Missed rays are returned as hit distance 0.0 and label 0.
- Object Observations: The agent receives the dimensions, relative positions, and velocities of up to 16 objects, sorted by the relative distance to the agent in the forward direction.
- Agent Velocity: XYZ vector representing the agent’s velocity.
- Vector from agent to goal: Agent-centric polar coordinate vector represent the direction and distance to the goal.
- Closest distance: a scalar representing the closest the agent has come to the goal in the current episode.
- Previous action: the agent’s previous taken action.

Agent actions are discrete 3-tuples  $a \in \mathcal{A} : \{0, 1\} \times \{0, \dots, 7\} \times \{0, 1\}$ , where the first component represents whether or not to move, the second component represents the agent relative movement direction, and the final component represents whether or not to jump.

To provide dense supervision to the RL optimization process, we use a reward function that gives a small reward on every time step that the agent decreases its minimum distance to the goal achieved thus far on the current obstacle course, as well as a large reward for completing the task by reaching the obstacle end goal. Our reward function for all obstacle courses is:

- Reward of  $+0.05 \times (\text{previous minimum distance} - \text{new minimum distance})$  each time a new minimum distance is achieved.
- Reward of  $+10$  (and the episode terminates) for touching the goal area with any part of the character.

We further facilitate learning by randomly sampling the agent position at the beginning of a training episode from safe regions

**ALGORITHM 1: Policy-Selection GoExplore**


---

```

1 Procedure ExplorePhase( $s_{start}$ ):
2   Input:  $s_{start}$  is the initial high-dimensional simulator state. Output:  $\mathcal{A}$  is the archive of all interestingly different bins and their best trajectories
   found.
3    $\mathcal{A} \leftarrow \{\text{bin}(s_{start}) \mapsto (s_{start}, R = 0, L = 0, \text{stats.init}())\}$  /*  $\triangleright$   $R$ : reward,  $L$ : trajectory length, bin: binning function */
4   while not Solved do
5      $c \leftarrow \text{SelectBin}(\mathcal{A})$  /*  $\triangleright$  Select bin using novelty-based sampling */
6      $s \leftarrow \mathcal{A}[c].\text{state}$  /*  $\triangleright$  Retrieve saved simulator state of bin  $c$  */
7     SetSimulatorState( $s$ ) /*  $\triangleright$  Restore exact state to avoid stochastic derailment */
8      $\text{policy} = \text{Random}$ 
9     if  $\text{stats.times\_chosen\_since\_update} == 0$  then
10      |  $\text{policy} = \text{PPO-Policy}$ 
11     end
12      $\text{stats.times\_chosen\_since\_update} \leftarrow \text{stats.times\_chosen\_since\_update} + 1$ 
13      $\text{stats.times\_chosen} \leftarrow \text{stats.times\_chosen} + 1$ 
14      $\mathcal{T}, \text{Solved} \leftarrow \text{Explore}(s, \text{policy}, k = 200)$  /*  $\triangleright$  Exploration for  $k$  environment steps */
15     foreach  $(s', r_{cum}, l_{traj}) \in \mathcal{T}$  do
16       |  $c' \leftarrow \text{bin}(s')$ 
17       | if  $c' \notin \mathcal{A}$  then
18         | |  $\mathcal{A}[c'] \leftarrow (s', r_{cum}, l_{traj}, \text{stats.init}())$  /*  $\triangleright$  Add newly discovered bin to archive */
19       | end
20       | else
21         | |  $(s_{old}, r_{old}, l_{old}, \text{stats}) \leftarrow \mathcal{A}[c']$ 
22         | | if  $r_{cum} > r_{old} \vee (r_{cum} == r_{old} \wedge l_{traj} < l_{old})$  then
23         | | |  $\text{stats.times\_chosen\_since\_update} \leftarrow 0$ 
24         | | |  $\mathcal{A}[c'] \leftarrow (s', r_{cum}, l_{traj}, \text{stats})$  /*  $\triangleright$  Update if higher reward or shorter trajectory found */
25       | end
26     end
27      $\mathcal{A}[c']. \text{stats.times\_seen} \leftarrow \mathcal{A}[c']. \text{stats.times\_seen} + 1$ 
28   end
29 end

30 Function SelectBin( $\mathcal{A}$ ):
31   Input:  $\mathcal{A}$  is the current archive of explored bins. Output:  $c$  is a sampled bin chosen proportional to its novelty score.
32   return  $c \sim \frac{1}{\sqrt{(\mathcal{A}[c]. \text{stats.times\_seen} + 1)}}$ 

33 Function BinningFunction( $s$ ):
34   Input:  $s$  is the high-dimensional environment state. Output: A low-dimensional discretized bin representation.
35    $x, y, z, \text{time}, \text{jumps\_used}, \text{interactive\_object\_pos} \leftarrow s$ 
36    $x\_bins \leftarrow 32$   $y\_bins \leftarrow 32$   $z\_bins \leftarrow 8$ 
37   if animated object then
38     |  $t\_bins \leftarrow 16$ 
39   end
40   if limited jumps then
41     |  $j\_bins \leftarrow \text{NumAllowedJumps}$  /*  $\triangleright$  If our queries constrain jumps, the limit is always less than 10. */
42   end
43   if interactive object then
44     |  $i\_bins \leftarrow 8$  /*  $\triangleright$  We bin only along forward (Y) axis of the obby. */
45   end
46   if animated object and limited jumps then
47     |  $x\_bins \leftarrow 16$   $y\_bins \leftarrow 16$ 
48   end
49   if animated object and interactive object then
50     |  $t\_bins \leftarrow 8$ 
51   end
52   if limited jumps and interactive object then
53     |  $z\_bins \leftarrow 4$ 
54   end
55   return  $(x\_bin, y\_bin, z\_bin, t\_bin, j\_bin, i\_bin)$ 

```

---

within the obstacle. Safe regions are defined as areas on top of blocks that do not damage the player.

### 3.4 Alignment to the Roblox Engine (the Target Game Engine Platform)

The implementation of movement mechanics in our obstacle course games take inspiration from the character motion in Roblox. For grounded movement in Roblox, the character rapidly accelerates to full movement speed and immediately decelerates when stopping. When jumping, the character can influence its movement in the air, which is essential for solving some obstacle course challenges like Lava Ring Drop (in which the solution involves using air mobility to avoid touching lava blocks).

Inspired by this, we implement character directional motion (and jumping) as an external force applied to the character on the timestep the action is taken. With the exception of the jump force, this force can be applied whether the character is in the air or on the ground. We implement air resistance to control the character's maximum speed. We tuned each of these values such that movement speed, jump height, and jump distance were closely matched between our obstacle course games and Roblox.

Using our high performance simulator, we were able to automatically align physics behaviors by matching character motion trajectories in our simulator to those in Roblox. Specifically, we recorded trajectories of agent actions and positions using the agent deployed in Roblox as discussed in the previous section. We then loaded identical geometry into our simulator, and searched over physics parameters (using 8192 worlds in parallel) in our simulator to find the combination that best aligned the resulting positions when the action recorded from the Roblox trajectories were replayed. This process was repeated until a certain error threshold across the entire trajectory was reached. The physical parameters we optimized for were gravity, directional movement force, jump force, and air resistance.

## 4 Strategies for State Selection and Exploration

In this section, we present ablations that address three questions: a) What are alternatives to our approach for using a pretrained policy during the exploration phase? (Sec. 4.1) b) Can we leverage a pretrained *encoder* (as in [Gallouédec and Dellandréa 2023]) to restore more promising states during the return phase? (Sec. 4.2) c) How do Reinforcement Learning (RL) based methods compare with GoExplore? (Sec. 4.3)

In the next two sections, we compare the methods on a wall-clock time budget of 5 minutes. This allows comparison of the methods based on sample efficiency, measured as the number of "Go" steps (ref.L5-1) where each go step performs 1.6M timesteps of simulation. We also compare solve rate (as described earlier) but without 90s time budget.

### 4.1 Alternative methods for combining RL Policy and GoExplore

We ablate the choice of exploration strategy after returning to a previous state. Our main strategy, which we find to be effective for longer queries, is to use the policy for duration of the exploration

step when the restored bin was new, referred to as GoExplore-PolicySelection (GE-PS) (Sec. 2.4). In this section, we explore two additional strategies: a) Random Selection: before every exploration phase (L15 in Alg. 1), choose either the pre-trained RL policy or the default random exploration, with 0.5 probability. This strategy is referred as GoExplore-RandomSlection (GE-RS) b) Only RL Policy: Always use RL policy within the GoExplore framework to sample actions, referred GoExplore-onlyRLPolicy (GE-RL).

In Fig. 2a, we find that all four strategies are solving nearly all the queries on Short and Medium regime. The differences emerge on Long queries: Policy Selection reaches a 0.56 solve rate, nearly doubling GE (0.31) and substantially outperforming both GE-RS and GE-RL (both at 0.38). If we compare GE-RS with GE-RL, both report identical solve rates and nearly identical mean steps to solve (Fig. 2b). While the former employs the policy for 50% of the times, the latter uses it all the time. The nearly identical results on both of these confirm that while using policy benefits the baseline GoExplore, it is still being used more times than necessary. Also, the pretrained policy is a low entropy policy which means using it more number of times (50% of GE-RS or 100% of GE-RL) will not give large variance in the actions. Thus the natural question is what is the minimum number of times could policy be used? Which motivates policy selection, which suggests to use it only once, when bin is newly sampled or updated. This frees up resources to run default random policy on number of times, which being a higher entropy policy, benefits from it by sampling diverse actions. This combination of pretrained policy with GoExplore gives 25% improvement in the solve rate in the long regime of queries.

### 4.2 Effect of Pretrained Encoder on Cell-Free Latent GoExplore

Cell-free Latent GoExplore (CFLGE) [Gallouédec and Dellandréa 2023] proposes to train an encoder to map discovered states to a latent representation during exploration. Within our 90s time budget, training this encoder from scratch did not produce a benefit in average solution time over GoExplore or our Policy-Selection (see Fig. 3, main paper). We therefore train an encoder offline, alongside the policy pretraining (ref. Sec. 3, using state-action-nextstate pairs from its rollouts. Following the CFLGE [Gallouédec and Dellandréa 2023] we use the forward dynamics objective to train the encoder. CFLGE-Frozen uses this pretrained encoder in place of the online encoder while running CFLGE, CFLGE-Finetune initializes from this pretrained encoder. We alternatively use CFLGE in place of CFLGE to highlight the encoder training differences with the frozen and finetune variants.

We find that in the short regime, CFLGE-Finetune improves the solve rate over the CFLGE by 5% while CFLGE-Frozen degrades its performance by 20% (Fig. 3a). In the medium regime, all of them have a comparable solve rates. We get the highest advantage of the pretrained encoder in the long regime where it improves the solve rate by 7x over CFLGE (21% from 3%). We hypothesize that the CFLGE and CFLGE-finetune's encoder overfits to the initial regions of the short queries. Since they are short horizon they are solved quickly as compared to using a frozen encoder which is suboptimal. When moved to long regime where the horizon increases, the CFLGE

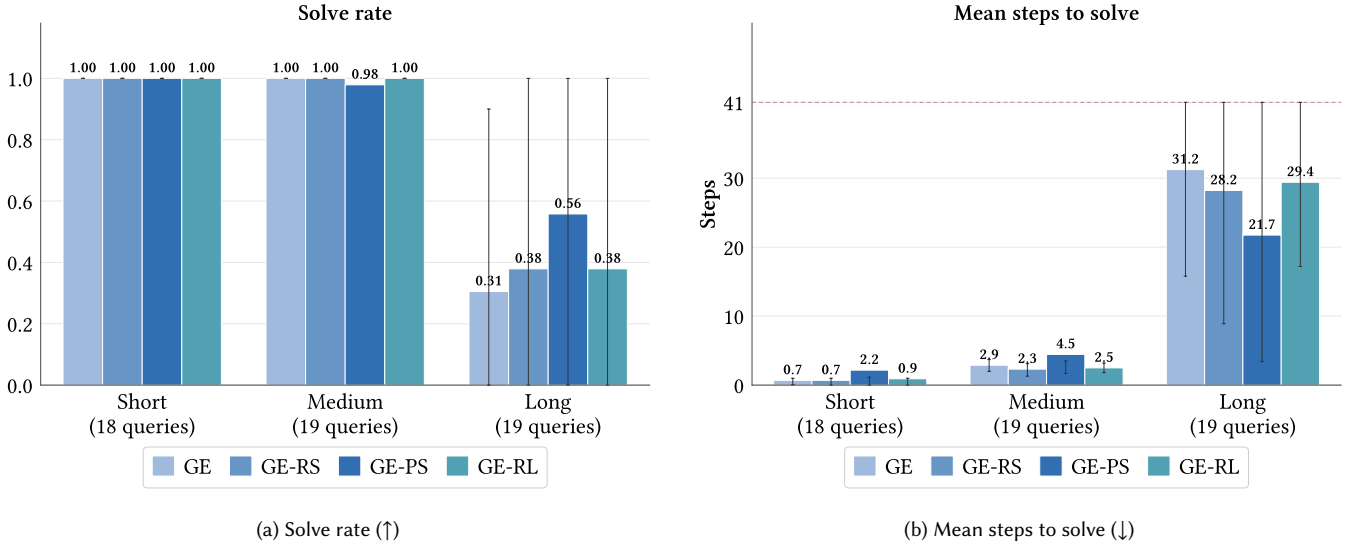


Fig. 2. Comparison of GoExplore variants on solve rate and sample efficiency (measured as GoExplore steps to solve): GoExplore (GE), GoExplore-RandomSelection (GE-RS), GoExplore-PolicySelection (GE-PS), and GoExplore-onlyRLPolicy (GE-RL)

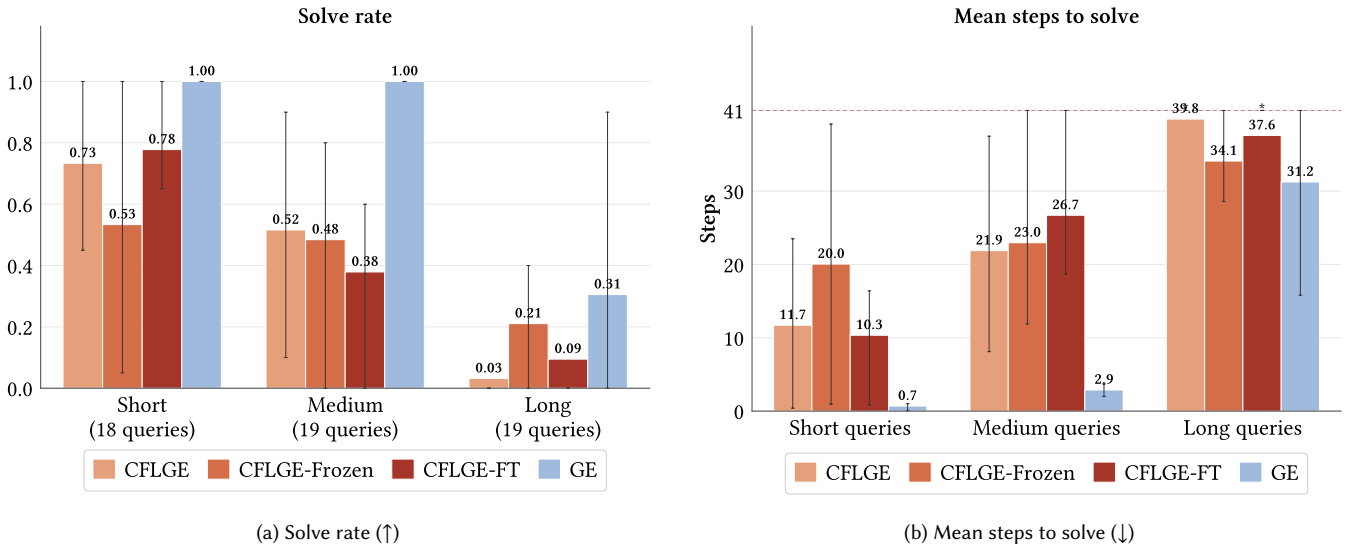


Fig. 3. Comparison of CFLGE variants against the GoExplore baseline. CFLGE-Frozen substantially outperforms (7x improvement in Solve Rate) CFLGE and CFLGE-Finetune on Long queries, but all lag behind vanilla GoExplore across all buckets.

and CFLGE-FT are unable to adapt to changing distribution of the states over the horizon and are stuck in local minima. While Frozen encoder has generalized representation their performance and all three variants have similar solve rates. CFLGE, despite starting from random weights, adapts its cells to whatever the current query looks like; CFLGE-Frozen is locked into a fixed encoding that may or may not generalize with a given Short query. While both offer similar solve rates for medium (48 and 52%), the benefits of frozen encoder are showcased in the long regime. In this regime CFLGE could solve only 3% of the times, which is boosted to 21% after using latents from

Frozen Encoder, which is a 7x improvement. This suggests that that on long queries, it is difficult to adapt the encoder on the streaming data while the pretrained latent space provides a meaningful prior having seen similar scenarios in the past. Fig. 3b shows the mean steps to solve over these methods, which inversely correlates with their solve rates.



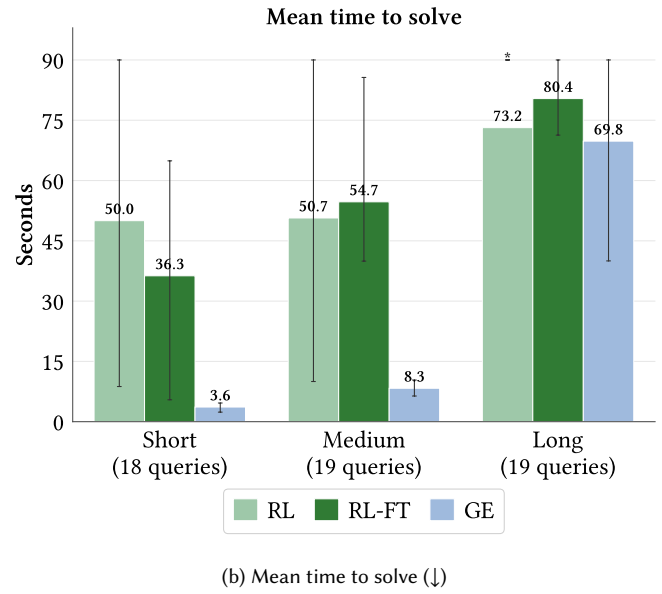
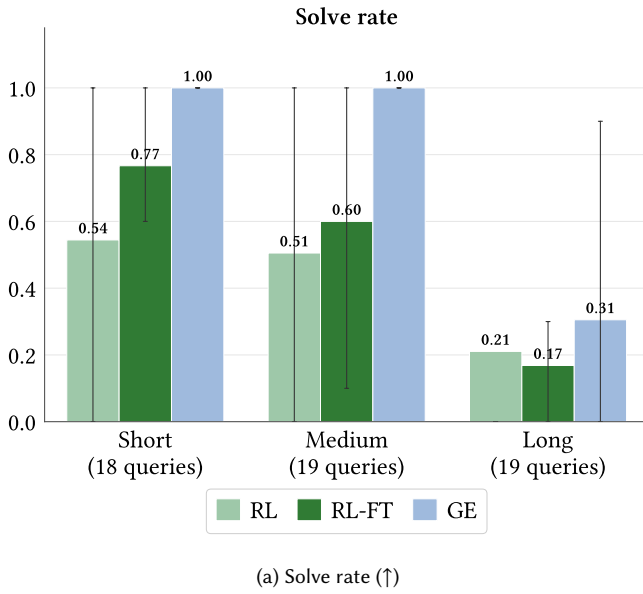


Fig. 4. Comparison of RL variants against the GoExplore baseline. RL-Finetuning outperforms RL Policy on solve rates, but all lag behind vanilla GoExplore across all buckets.

### 4.3 Comparison of RL Policy with GoExplore

We compare the two variants of reinforcement learning-based methods in Fig. 4. In the first variant, we pretrain a PPO [Schulman et al. 2017] policy, as detailed in Sec. 3. We then run inference on this policy with no GoExplore search on our query set as many times as possible under 90s time budget until it solves for a maximum of 5 times and report solve rate (out of 5) and mean times to solve. We refer to this variant as “RL Policy”. In the second variant, we finetune this policy while running it on each of our queries by providing the reward signal detailed in Sec. 3 for each step of the rollout. We refer to this variant as “RL Finetuning”. Any rollout reaching the goal during finetuning is counted as a success. We finetune across 5 seeds and report the average solve rate and mean time to solution.

On solve rates (Fig. 4a), RL Finetuning outperforms RL Policy on Short queries (0.77 vs 0.54). Both have similar performance in Medium regime, while in Long regime finetuning slightly degrades the solve rate (0.17 vs 0.21). This is because gradient updates can speed up the solution search in short-term for the policy (controlled by discount factor of PPO) which benefits the short regime queries. But it is a myopic update for the long horizon queries leading the agent to deceptive regions and derail its learning. This causes degradation in solve rate as compared to RL Policy inference. We also find that in the long regime the RL Policy can solve only those queries which are similar to the template queries that it encountered in the pretraining. In Fig. 4b, we show the mean time to solve across these methods. The trend on the solve rates is correlated with mean time, where in short regime the RL Finetuning is faster than RL Policy, in medium regime they are comparable, while in the long regime RL Finetuning slightly degrades the performance of RL Policy. In all the regimes, we have also compared with GoExplore which outperforms

over all the regimes and both the metrics (solve rate and mean time to solve).

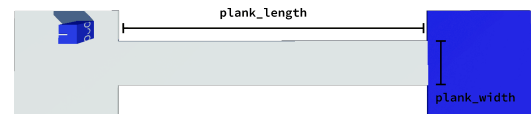
## 5 Procedural Obstacle Game Environment Generation

We train our policy on procedurally generated environments. Our procedurally generated obstacles were designed to model common design patterns and idioms seen in published obstacle course games on popular game platforms (e.g., traversing long walkways, jumping between blocks with air gaps in between, jumping over a wall).

Our procedural system organizes design patterns into a fixed set of parameters obstacle segment types. Below is a description of each parameterized obstacle type. For each, we include its parameters and a visualization of how those parameters correspond to the realized obstacle.

### Plank

A plank is a straight platform from the spawn point to the goal.



**plank\_width:** float

Describes the width of the plank.

**plank\_length:** float

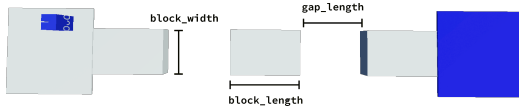
The length of the plank which a player must cross.

### Path Jumps

A series of identical blocks uniformly spaced apart in a straight line to the goal.

**num\_blocks:** int

The total number of blocks.



`block_width: float`

The width of each block.

`block_length: float`

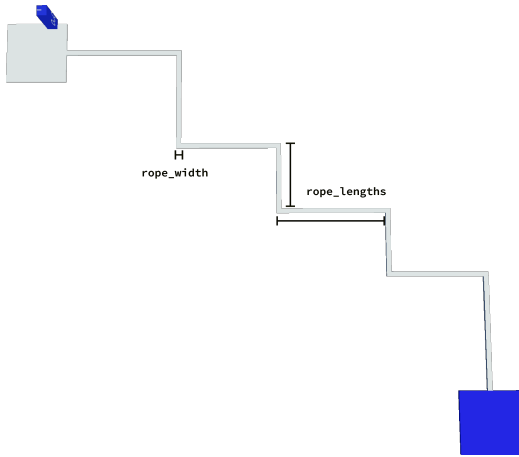
The length of each block.

`gap_length: float`

The size of the gap between each block.

### Tightrope

A sequence of zigzagging platforms leading to the goal.



`num_ropes: int`

The number of rope segments.

`rope_lengths: list[float]`

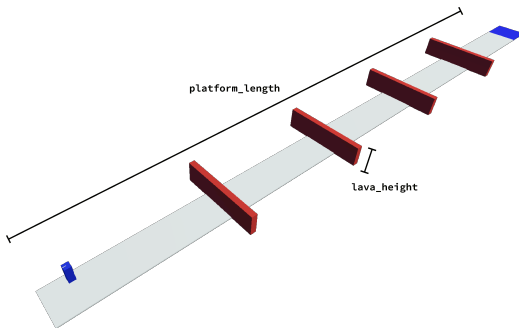
The length of each rope segment.

`rope_width: float`

The width of the rope segments.

### Lava Hurdles

A platform with evenly spaced lava blocks of some height. The player must jump over the blocks to get to the goal.



`num_hurdles: int`

The number of lava hurdles.

`lava_height: float`

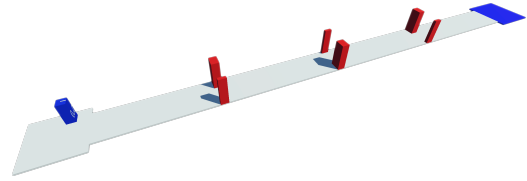
The height of each lava hurdle.

`platform_length: float`

The total length of the platform.

### Lava Doors

A platform with two lava columns on the edges. The player move through the opening.



`platform_length: float`

The total length of the platform.

`platform_width: float`

The total width of the platform.

`lava_height: float`

The height of each lava column.

`door_widths: list[float]`

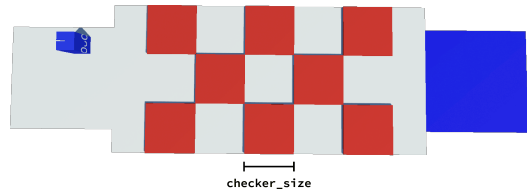
The widths of the opening between each door.

`door_widths: list[float]`

The position of the center of each opening (as a fraction of platform\_width)

### Lava Checkerboard

A checkerboard three squares wide and variable number of rows long. The checkerboards alternate between regular squares and lava squares.



`num_rows: int`

The number of checkerboard rows.

`checker_size: float`

The size of each checkerboard square.

### Wall Wraps

A series of walls which require the player to maneuver around the walls in the air.



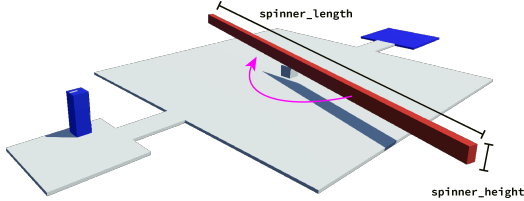
`num_walls: int`

The number of walls to jump around.

799 wall\_thickness: float  
800 The thickness/depth of each wall.

801 **Lava Spinner**

802 A square platform with a one or two rotating lava spinner at the center.  
803



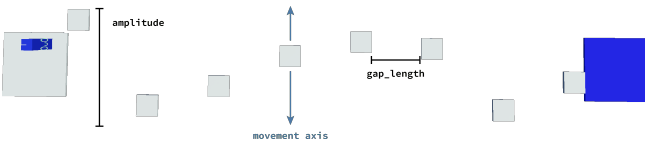
813 num\_spinners: (1,2)  
814 The number of lava spinners.  
815 platform\_size: float  
816 The dimension of the square platform.

817 Each spinner has its own:

818 spinner\_height: float  
819 The heights of the lava spinner.  
820 spinner\_length: float  
821 The length of the lava spinner (diameter).  
822 spinner\_thickness: float  
823 The width of the lava spinner.  
824 angular\_velocity: float  
825 The speed at which the lava spinner rotates.  
826 spinner\_position: float  
827 The position of the center of the spinner with respect to the  
828 platform.  
829 starting\_orientation: float  
830 The orientation of the lava spinner at the start of an episode.  
831

832 **SideToSideBlocks**

833 A series of horizontally moving blocks.  
834



842 num\_cases: int  
843

844 **Wedges**

845 A straight platform with wedges across the platform.  
846

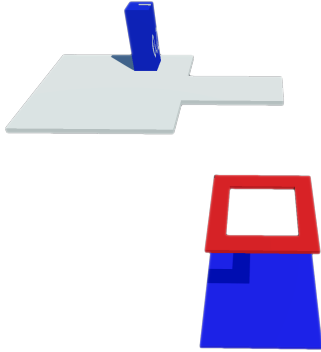


851 num\_wedges: int  
852 The total number of wedges.  
853 wedge\_length: float  
854 The length of each wedge.  
855

856 wedge\_height: float  
857 The height of each wedge at its peak.

858 **Lava Ring Drop**

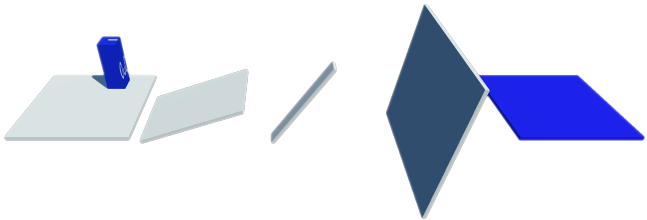
859 A ring of lava positioned above the goal.  
860



861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876 goal\_size: float  
877 Side length of the goal.  
878 goal\_position: (float, float, float)  
879 Position of the goal.  
880 ring\_border\_thickness: float  
881 Width of the lava ring border.  
882 ring\_hole\_size: float  
883 Side length of the lava ring hole.  
884 ring\_position: (float, float, float)  
885 Position of the lava ring  
886

887 **Rotating Platforms**

888 A series of three rotating platforms that the player must cross to  
889 reach the goal.  
890



891 Each platform has its own:

892  
893  
894  
895  
896  
897  
898  
899  
900  
901 size: (float, float)  
902 Length and width of the platform  
903 angular\_velocity: float  
904 Angular velocity of the platform  
905 starting\_orientation: float  
906 Starting orientation of the platform at the start of the episode  
907

908 **Staircase**

909 A series of steps and platforms.  
910  
911  
912



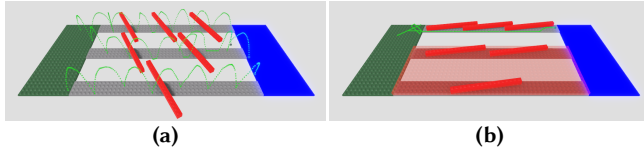
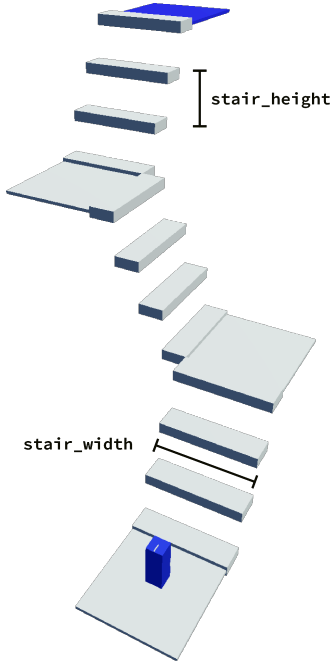


Fig. 5. Our query primitives enable verification of multiple solution paths. (a) Solutions along all three paths of the obstacle are verified simultaneously. (b) Using an avoid region with a jump count allows the designer to verify that the most difficult path is traversable without jumping.



```
num_cases: int
    The number of chained staircases.
num_stairs: int
    The number of stairs per staircase.
stair_height: float
    The size of the vertical gap between each stair.
stair_width: float
    The width of each stair.
```

## 6 Additional Design Case Studies

In this section we document addition design case studies that were not included in the main paper.

### 6.1 Additional Case Study 1: Evaluating multiple solution paths

The goal in this design is to create an obstacle with three paths featuring increasing difficulty of the same type of spinning lava block challenge. After creating the initial design, the designer wishes to know if solutions exist down each of the three paths and what those solutions look like.

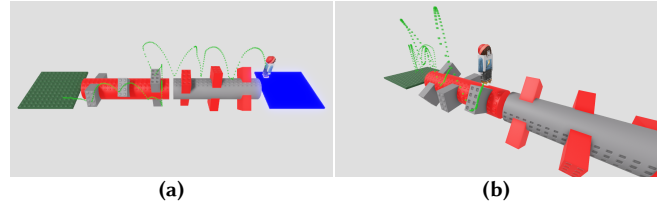


Fig. 6. Feedback from our system supports analysis of an extremely difficult obstacle. (a) The obstacle moves too fast to support the designer's intended solution, (b) moderate speed allows waiting.

The designer queries to see if the level is solvable along each of the intended paths. Our system returns multiple solutions that together demonstrate that a solution along each path exists (Fig. 5, (a)), answering the first part of the designer's question. The trajectory visualizations further show that each path makes a lot of jumps. The designer now wishes to know which of these jumps are necessary for a solution.

To start answering this question, the designer queries for solutions with no jumping. The query shows that at least one path can be traversed without jumping at all. More queries confirm that this is true for all three paths (most difficult path visualized in Fig. 5, (b)). This demonstrates that, surprisingly, the answer to the designer's question is: no jumps are necessary for a solution. This is a counter-intuitive result that leads to interesting gameplay because it allows for less precise motions at the cost of much more precise timing. So, instead of attempting to edit the level to force jumping, the designer decides to leave the level as is.

The tool allowed the designer to verify properties of their initial design and revealed unexpected properties that changed the course of the design process. Note that it was only after seeing the first round of solutions that the designer formulated their refined queries—they did not know at the start that a solution without jumping was possible or desirable.

### 6.2 Additional Case Study 2: Evaluating a difficult obstacle

The designer sets out to create an extremely difficult obstacle that will likely require practice over multiple attempts to solve consistently. Precise motion and timing will be required to stay on the spinning spikes in the first half and then transition to avoiding them in the second half.

The designer does not know at start if a solution exists or how to analyze it. They first query to see a solution. The system returns that a solution *does* exist. The designer notices that the solution trajectory briefly waits on one of the initial spikes before jumping to the next, and hypothesizes that increasing the speed might make this impossible, and therefore make the obby impossible to solve. Another query tests this hypothesis.

The result is that the obstacle is still solvable—that is, while it may be impossible to wait on a spike, this fact does not make the obby impossible to solve (Fig. 6, left). Despite this, a view from the player camera causes the designer to conclude that the spikes are spinning too fast for a player to reason about. So, the designer edits

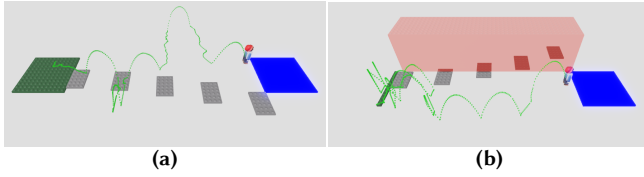


Fig. 7. Our query primitives can express queries that answer questions about complex gameplay. (a) The system verifies the first solution the designer envisioned. (b) Using our query primitives, the designer verifies that the second, more fluid solution is possible.

the speed back down, this time to even slower than the original speed.

The designer now wishes to know if solutions that feature waiting on an initial spike exist. Another query shows that the agent can spend significant time (at least half a second) waiting on an intermediate spike to jump to a final one (Fig. 6, right). This is challenging and in line with the designer’s goals, so it is the final design.

In this exercise, our system showed a solution even though the designer did not have an intended solution in mind or know how to reason about what properties of a solution helped or hindered their goals. After seeing a solution, they were able to do so even for this extremely difficult obby that they could not reliably playtest by hand.

### 6.3 Additional Case Study 3: Answering complex gameplay questions with queries

In this design, the designer intends the player to jump between a series of faster and faster oscillating blocks. The designer intends that the player should have time on each block to line up their jump if necessary. But, the obstacle should also be solvable in a faster, more fluid way, without as much waiting on slower blocks for the faster blocks to align—that is, the blocks should line up if the player starts crossing at the right time.

The designer queries to see if the blocks are correctly spaced and not moving too fast to jump to. The solution visualization says that both these properties are true: the player is able to jump between all blocks, and can wait on the third block before jumping to the fourth (Fig. 7, left).

The designer now tests to see if the more fluid solution is supported. Specifically, the designer wants to know: are the blocks moving in such a way that the player can traverse the obstacle staying on only one side of the center line? To answer this question, the designer sets up a query that forces solutions to avoid roughly half the span of each moving block. This query shows a solution that crosses nearly completely on one side of the centerline, and as a result features much less waiting on intermediate blocks (compare the length of trajectory spent standing on the third block in Fig. 7 left vs. right). This design meets the designer’s goals.

Here, our system assisted the designer by allowing them to analyze a complex gameplay property. The query used to test this gameplay property is one which, without policy selection (Sec. 2), often does not find a solution, which might give misleading feedback to the designer.

### 6.4 External Designer 3

The designer articulated a starting goal of “something like running down a hallway with things flying at you”. As a first pass, they started with a thin walkway and added spinning lava obstacles to it (Fig. 8, (a)).

The designer’s initial goal was to find the point at which the obstacle could not be solved and “work backwards” from there. They ran a query to determine if their initial configuration was solvable, and found that a solution existed but was too difficult because of the high number of jumps over lava obstacles. Next, the designer wondered if it would be easier to ride a larger, slower spinner to a faraway goal instead of avoid smaller, faster spinners.

After modifying the design, the designer made two queries. First, they verified that it was possible to climb the starting walkway to get to a “pickup point” for the spinner (Fig. 8, (b)). Second, they verified that it was possible to ride the spinner from the pickup point to the goal (Fig. 8 (c)). Finally, the designer suspected that it should be possible to jump to the larger spinner from the starting block without going along the walkway. Using a query, they confirmed that this exploit exists (Fig. 8, (d)).

As with Designer 1, Designer 3 stated that the most important feedback was looking at the trajectories (rather than learning whether a design was solvable). This was particularly important even if the designer ended the search before a successful trajectory was found—often, the initial visualization told them what they needed to know.

### References

- Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. 2021. First return, then explore. *Nature* 590, 7847 (Feb. 2021), 580–586. <https://doi.org/10.1038/s41586-020-03157-9>
- Quentin Gallouédec and Emmanuel Dellandréa. 2023. Cell-free latent go-explore. In *Proceedings of the 40th International Conference on Machine Learning (Honolulu, Hawaii, USA) (ICML’23)*. JMLR.org, Article 426, 16 pages.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *CoRR* abs/1707.06347 (2017). [arXiv:1707.06347](https://arxiv.org/abs/1707.06347)

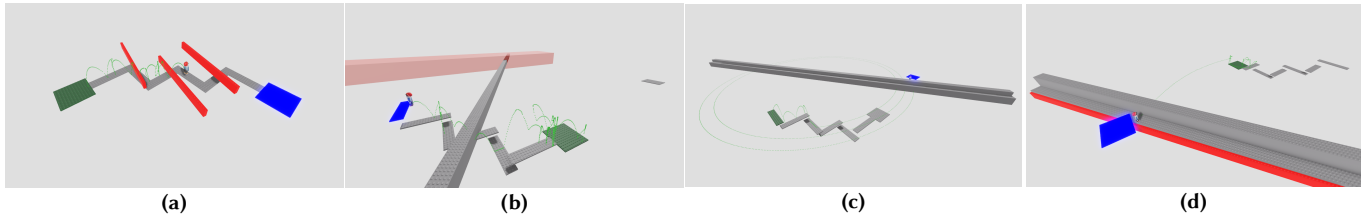


Fig. 8. External Designer 3: This designer used the tool to verify solutions while creating a complex obstacle. **(a)** Feedback from the system verified that the designers initial design was solvable, but they decided it was too difficult because of the number of jumps over lava spinners required. **(b)** In the next design iteration, our system verifies that the player can solve the obstacle with a large spinning rod sweeping periodically over the space. **(c)** On the same obstacle, the designer verifies that, once the player has reached the end of the path, they can catch the spinning rod and be swept to the goal. **(d)** The designer verifies that an exploit exists that skips walking the path shown in (b).